

COMPUTER NETWORKS

Complete Course Notes with Layer Diagrams & Protocol Tables

Definitions + OSI/TCP-IP Diagrams + Protocols + Subnet/CRC/Dijkstra Worked + Tools | 5 Units

SYLLABUS COVERED

Unit 1 Introduction & uses; LAN/MAN/WAN, topologies; protocol hierarchies & design issues; OSI & TCP/IP reference models, services/interfaces, PDUs, encapsulation	Unit 2 Data link layer: framing, error detection (parity, CRC), flow control (stop-and-wait, sliding window, go-back-N, selective repeat), ARQ; Ethernet frame & addresses	
Unit 3 MAC sublayer: ALOHA/slotted ALOHA, CSMA, CSMA/CD, IEEE 802.3 Ethernet, 802.11 Wi-Fi, switches/hubs/VLANs, ARP/RARP, LLC	Unit 4 Network layer: routing (flooding, distance-vector Bellman-Ford, link-state Dijkstra), IP IPv4 header, classful addressing & classless CIDR, subnetting worked, fragmentation, ICMP, IPv6	
Unit 5 Transport layer: UDP vs TCP, TCP segment fields, 3-way handshake, timers, flow control, congestion control (slow start / AIMD); application layer: DNS, HTTP/HTTPS, SMTP/POP/IMAP, FTP & the tools to see them	Practical Every unit has a working shell/toolbox line (tcpdump, ip, dig, curl, nc, ss)	How to use Read in order; redraw the OSI stack + TCP handshake; redo subnet/CRC by hand

Self-study & exam-prep guide following the CN syllabus. All diagrams are vector graphics - crisp and colourful on screen and in print.

TABLE OF CONTENTS

Introduction, Reference Models & Physical Layer	uses · L switching
Data Link Layer - Framing, Error & Flow Control	framing
MAC Sublayer & IEEE 802 LANs	ALOHA
Network Layer - IP Addressing & Routing	datagram ICMP · IP
Transport & Application Layers	UDP vs

Study tip: redraw the OSI/TCP-IP stack and the TCP handshake from memory; redo one subnet + one CRC division before the exam.

UNIT 1 | INTRODUCTION, REFERENCE MODELS & PHYSICAL LAYER

1.1 Why Networks & Categories

NETWORK BASICS

A network interconnects devices so they can SHARE resources (files, printers, CPU) and COMMUNICATE. Categories by size: LAN (1 building, ethernet/Wi-Fi), MAN (city, metro-ethernet), WAN (country/world, routers + leased lines). Topology: bus, star, ring, mesh, tree - each trading cable cost, reliability and failure blast radius.

Topology	Cable cost	Failure effect	Used by
Bus	low	whole segment dies on one break	legacy coax ethernet
Star	med (hub)	that link only	modern LAN (switch at centre)
Ring	med	two-way ring can route around	FDDI, token ring
Mesh	high	rerouting saves it	WAN backbones, cloud

1.2 Protocol Hierarchy & Reference Models

LAYER PROTOCOL DESIGN

A PROTOCOL = agreed rules (syntax, semantics, timing). LAYER N offers a service to N+1 through the SAP (service access point); each layer adds its OWN HEADER, so the same message has many encapsulated forms on the wire - the "onion" model. Design issues repeated at each layer: addressing, error control, flow control, multiplexing/demultiplexing, fragmentation.

OSI 7-Layer vs TCP/IP 5-Layer Stacks

7 APPLICATION	Application	HTTP, DNS, SMTP, FTP
6 PRESENTATION		TCP / UDP
5 SESSION		IP + routing (IPv4/IPv6)
4 TRANSPORT	Transport	Ethernet, Wi-Fi, ARP
3 NETWORK	Network	cables, radio, fibre
2 DATA LINK	Data Link	
1 PHYSICAL	Physical	

OSI = reference taught; TCP/IP = the stack actually deployed on the internet

PDUs: application -> message, transport -> segment, network -> packet, link -> frame, physical -> bits

OSI (generic) maps onto TCP/IP (deployed); 1-4 = actual transport of bits

Layer	Job in 10 words	Real protocol
Application	what the user's program asks	HTTP, DNS, SMTP, FTP
Presentation	translate codes, encrypt	(merged into app in the real internet)
Session	keep a conversation alive	(rarely a separate layer today)
Transport	reliable end-to-end data	TCP, UDP
Network	route packets across networks	IP, ICMP
Data link	frame + error-free hop delivery	Ethernet, Wi-Fi, PPP
Physical	bits over the cable/radio	10/100/1000BASE-T, OFDM

OSI VS TCP/IP - EXAM COMPARISON

OSI separated presentation/session/transport; TCP/IP merged them and won by BEING deployed first. TCP/IP stack: network interface / internet / transport / application. Encapsulation (wrap) + decapsulation (unwrap) at the far end; each layer sees peer-to-peer protocol with its opposite number.

layers.txt

```
# the onion, from a browser that just loaded a page:
packet: [Eth][IP][TCP][HTTP: "GET / HTTP/1.1" ...][TCP][IP][Eth]
        L2  L3  L4  L7 (payload)          L4  L3  L2
# tools that read the layers:
tcpdump -i eth0 -nn port 80      # show Frames (L2) + IP (L3) + TCP (L4)
curl -v http://example.com/      # shows the HTTP request line (L7)
ip addr ; ip route               # configure L3 addresses
```

1.3 Physical Layer

SIGNALS & MEDIA

Data -> encoded as bits -> transmitted by energy: GUIDED media = twisted pair (cheap, common), coaxial (older broadband), optical fibre (tens of km, high speed, immune to EM noise); UNGUIDED = radio/Wi-Fi, microwave links, satellite. Bandwidth vs data rate: more bandwidth => more bits; Shannon limit $C = B \log_2(1+S/N)$.

MULTIPLEXING

Frequency-division (separate frequency bands - radio stations), time-division (fixed slots - DSL/TDM), wavelength (optical). Switching: CIRCUIT SWITCHING (reserved path, phone - poor for bursty computer traffic), PACKET SWITCHING (store-and-forward, statistical sharing - the internet), MESSAGE switching (one big store-forward unit).

CIRCUIT VS PACKET SWITCHING + A PHYSICAL-LAYER REALITY CHECK

circuit: dial -> reserve -> talk -> release (guaranteed delay, wasted when idle)
packet: each packet routed independently (burst-friendly, best effort, delay varies)
Ethernet example: cat5e cable ~ 100 m, 1 Gbps; fibre LC/SC links run 10-100+ Gbps over km
practical check: `ethtool eth0 | head ; iwconfig ; tracepath 8.8.8.8`

DESIGN ISSUES

QoS duty: throughput, delay, jitter, loss - networks trade them (best effort vs guaranteed).

Exercise: 1) Map the 7 OSI layers to the 5 used by TCP/IP with one protocol each. 2) Draw encapsulation for an HTTP request (labels Eth/IP/TCP/HTTP). 3) Compare circuit vs packet switching in 3 points. 4) Shannon: $B=3\text{kHz}$ $S/N=1000$ -> C ? (29.9 kbps).

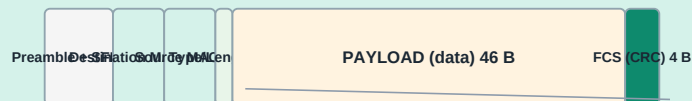
UNIT 2 | DATA LINK: FRAMING, ERROR & FLOW CONTROL

2.1 Data Link Layer Job

WHAT THE DLL DOES

Turns a raw bit stream into FRAMES: framing (delimit units), error detection (CRC), flow control (don't drown the receiver), error recovery (ARQ resends), and (Ethernet) access control. Between two ADJACENT machines - one hop.

Ethernet Frame (IEEE 802.3) - Layer 2 Packet



minimum frame 64 B (preamble excluded); max 1518 B; FCS = CRC-32 over header+payload

switch learns: records (source MAC -> port); ARP needed to map IP->MAC before ethernet can carry it

the 802.3 frame: addressing, type, payload, integrity tail

FRAMING METHODS

Character counting (count field; one corrupt count = desync), flag bytes with byte stuffing (escape special bytes), bit stuffing (insert 0 after five 1s), and physical-layer coding violations (fancy PHY, e.g. in Manchester).

2.2 Error Detection & Correction

Code	Detects	Corrects	Cost
Parity (1 bit)	odd number of flipped bits	no	1 bit/word
Checksum (TCP/UDP, header)	most single-word corruption	no	16-bit complement
CRC-32 (Ethernet)	bursts, up to ~32 bits	no	32 bits/frame
Hamming (7,4)	2-bit errors	1-bit errors	3 parity bits/4 data

●●● crc.txt

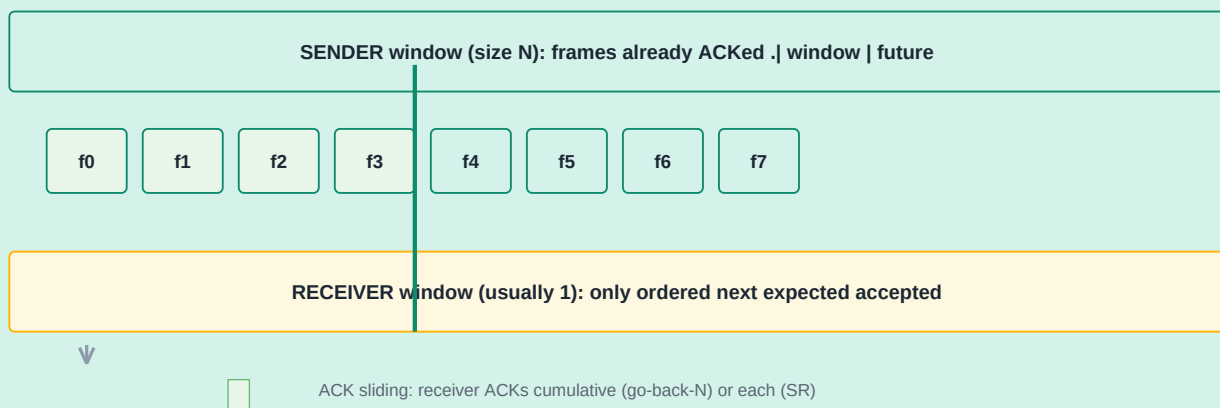
```
# CRC the way a computer does it (mod 2):
message M(x)=11010011101100 divisor G(x)=x^4+x+1 = 10011 (degree 4 -> 4 CRC bits)
1) append 4 zeros -> M'
2) XOR divide by G; remainder = FCS
3) send M + FCS such that (M+FCS)/G leaves ZERO
# receiver just re-divides: remainder==0 => accept
# polynomial: CRC-32 = x^32+x^26+x^23+x^22+x^16+x^12+x^11+x^10+x^8+x^7+x^5+x^4+x^2+x+1
```

2.3 Flow Control & Sliding Window

STOP-AND-WAIT VS SLIDING WINDOW

SEND-AND-WAIT: one frame at a time - tiny sequence numbers (1 bit) but line idle waiting (efficiency $\sim 1/(1+2a)$, a = propagation/transmission). SLIDING WINDOW: keep N frames in flight, ACK advances the window - pipelining fills the pipe. Sender window size must be $\leq 2^{(m-1)}$ for go-back-N (m = sequence bits) to avoid ambiguity.

Sliding Window & ARQ: go-back-N vs selective repeat



go-back-N: one damaged/ lost frame -> resend window from it; selective repeat: resend only the hole

windows give pipelining (many in flight) + the two ARQ recovery policies

ARQ scheme	Loss handling	Window	Bandwidth use
Stop-and-wait	resend 1 frame on timeout	1	poor on long links
Go-back-N	resend from the failed sequence	N	good, wasteful in bursts
Selective repeat	resend ONLY lost; receiver buffers out-of-order	$\leq 2^{(m-1)}$	best, needs big buffer

TRICK FACT

1-bit window = alternating-bit protocol (the special case examiners love).

Exercise: 1) Frame the string with bit stuffing around a flag "0x7E". 2) Will CRC catch a 2-bit burst? 3) Data rate 1 Mbps, propagation 50 ms, frame 1000 bits: efficiency of stop-and-wait? 4) Why must window $\leq 2^{(m-1)}$ in go-back-N?

UNIT 3 | MAC SUBLAYER & IEEE 802 LANs

3.1 Random Access (MAC) protocols

HOW A SHARED CHANNEL IS SHARED

The MAC sublayer sits above the DLL to decide WHO talks on a multi-access channel. Random-access family: ALOHA and slotted ALOHA (transmit anytime / in slots - max efficiency 18.4% vs 36%), CSMA (listen first), CSMA/CD (collision detection + backoff - ethernet), CSMA/CA (collision avoidance - Wi-Fi, can't listen while sending).

Protocol	Rule	Max throughput	Where
Pure ALOHA	send whenever, resend on collision after random wait	18.4% ($1/2e$)	early packet radio
Slotted ALOHA	send only at slot start	36.8% ($1/e$)	discrete-time lab
1-persistent CSMA	listens; when idle send; on busy keep waiting	~ near CD cap	degenerate
p-persistent CSMA	with prob p send in idle slot	tunable	theory
CSMA/CD (ethernet)	detect collision, jam, binary-exponential backoff	~ ($1/(1+6.44a)$)	IEEE 802.3

CSMA/CD TIMELINE + THE MINIMUM-FRAME ARGUMENT

CSMA/CD packet timeline: carrier-sense -> transmit -> listen WHILE sending
 collision window = $2 \times$ propagation delay; slot time = 512 bit times (10 Mbps)
 after collision: wait random $0..2^k-1$ slots (k = retry, capped) -> BEB
 why ifp: detecting while sending requires MIN frame $\geq 2 \times$ worst-case diameter

KERNEL INSIGHT

The MINIMUM FRAME mystery: frame too short vs long wire => transmitter finished before collision comes back. This derives slot time = $2 \times$ TPD.

3.2 IEEE 802 LAN family

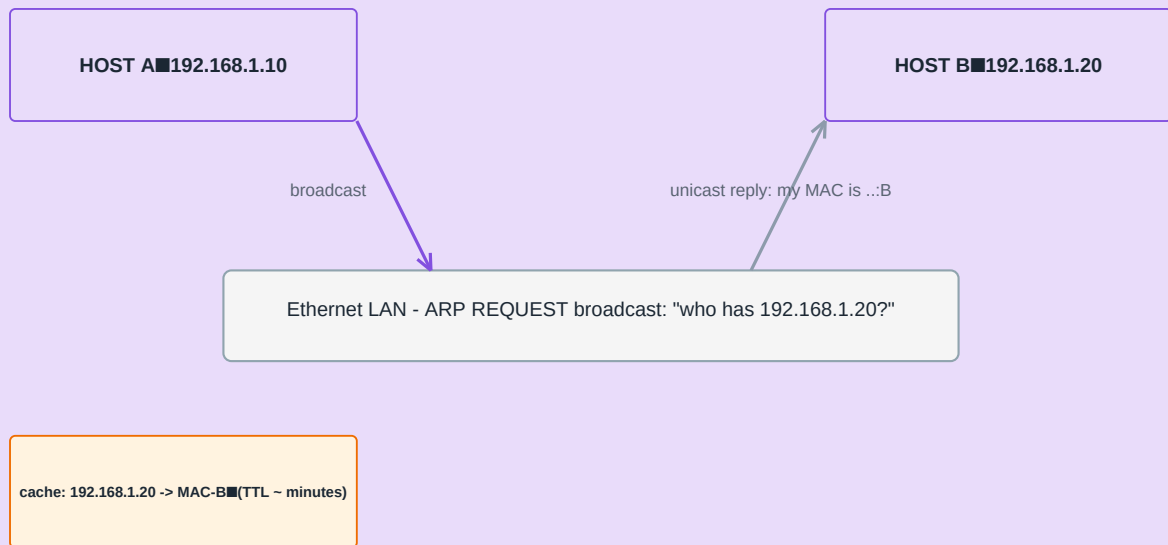
Standard	Name	Speed / medium
802.3	Ethernet (CSMA/CD / full duplex)	10/100/1000M, 10G/100G fibre+copper
802.11	Wi-Fi (CSMA/CA)	a/b/g/n/ax (11ax ~ up to 9.6 G)
802.1D/Q	bridges / VLAN tagging	switching fabric
802.15 / BLE	WPAN (bluetooth)	~2 Mbps
802.16	WiMAX (metro wireless)	historic

LAN ANATOMY: HUB BROADCAST VS SWITCH FORWARDING

Ethernet frame recap: [Preamble|DA|SA|Type/Len|Payload 46-1500|FCS-32]
 switch vs hub: hub = repeater/broadcast, ONE collision domain; switch = frames to the right port
 switches learn MAC->port; loops broken with spanning tree (802.1D); VLANs isolate traffic

3.3 ARP/RARP & LLC

ARP: how IP becomes Ethernet MAC



RARP = reverse (diskless host: give me MY IP from MAC); DHCP now does both jobs

ARP request is broadcast at layer 2; the reply is unicast - then IP is carried by ethernet

ARP INSIDE THE STACK

IP addresses are layer-3; Ethernet needs layer-2 MAC. ARP (Address Resolution Protocol) maps IP->MAC with a broadcast request + unicast reply, cached with a TTL. RARP (obsolete) was the reverse: diskless hosts ask "what is MY IP?" - now DHCP does it. LLC (802.2) is the tiny sublayer giving SAP services to the network layer over any 802 MAC.

WHERE ARP SITS: IP-TO-MAC RESOLUTION BEFORE ANY IP FRAME FLIES

```

layer3  ARP table: 192.168.1.20 -> 00:1a:2b:3c:4d:5e (renew every few minutes)
layer2  ethernet source/dest = the two MACs; IP lives ONLY inside the payload
ping:   ICMP over IP over ethernet; traceroute: UDP/TTL ticks from router to router
  
```

Exercise: 1) Compute pure vs slotted ALOHA max efficiency. 2) Why is CSMA/CD impossible on Wi-Fi? 3) Trace a ping from A(192.168.1.10) to B(192.168.1.20) naming every protocol (ARP, Ethernet, IP, ICMP). 4) Hub vs switch - how many collision domains each?

UNIT 4 | NETWORK LAYER: IP ADDRESSING & ROUTING

4.1 Network Layer Design & IP

NETWORK LAYER IN ONE LINE

Services: connectionless (datagram, IP - every packet routed alone) vs connection-oriented (virtual circuit). It addresses hosts globally (IP addresses), fragments big packets, routes through ISPs/routers, and reports errors (ICMP).

Field block	IPv4 (32-bit) feature	IPv6 (128-bit) feature
Addressing	dotted decimal, classes A-E, NAT needed	hex colon form, earth of addresses, no NAT
Header	variable, options, checksum	fixed 40 B, no checksum, extension headers
Broadcast	255.255.255.255 = limited broadcast	multicast + unicast preference
Auto-config	DHCP	SLAAC (stateless) or DHCPv6

```
ip_practical.txt
```

```
# see your own L3 identity on any OS:
ip addr show          # linux:      inet 192.168.1.10/24
ip route              # default via 192.168.1.1 dev eth0
# windows: ipconfig /all ; Get-NetIPAddress
# the default gateway = your router's IP - L3 hop #1
```

4.2 Classful Addressing & Subnetting

CLASSES & CIDR

Class A: 0/8 (huge), B: 10/16, C: 110/24, D multicast, E research. Classful wasted addresses, so CIDR (slash notation) + VLSM took over: 192.168.1.0/24 means subnet mask 255.255.255.0. Subnetting: borrow host bits to make SUBnets. PRIVATE ranges (RFC 1918): 10.x, 172.16-31.x, 192.168.x - NAT hides them behind one public IP.

```
subnet_drills.txt
```

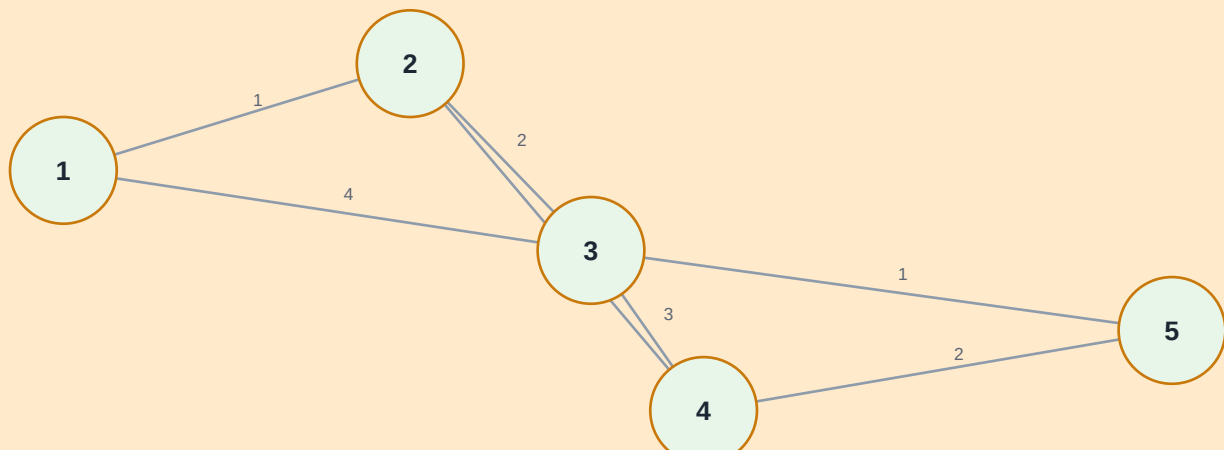
```
# keep the 3 calculation drills sharp:
mask /24 -> 256 addresses, 254 usable (2 reserved: network + broadcast)
subnets: /24 into /26 = 2^2 = 4 subnets x 64 addresses -> 62 usable each
Q: 192.168.4.0/26 - list the 4 subnet addresses: .0, .64, .128, .192
Q: host 10.10.10.65/26 - network = 10.10.10.64, broadcast = .127, range .65-.126
```

4.3 Routing Algorithms

ROUTING FAMILIES

STATIC (admin-set) vs DYNAMIC (learn topo): DISTANCE-VECTOR (Bellman-Ford - neighbours swap tables; RIP hop metric; count-to-infinity disease), LINK-STATE (flood state + Dijkstra; OSPF), and for backbones BGP (path-vector between ASes). Round arounds: flooding (no tables, broadcast storms) and multipath EQ.

Link-State Routing: Dijkstra from node 1



iter dist[1] dist[2] dist[3] dist[4] dist[5] done-set

init 0 1 4 8 9 {1}

1 0 1 3 3 7 {1,2}

2 0 1 3 3 4 {1,2,3}

3 0 1 3 3 4 {1,2,3,4}

4 0 1 3 3 4 {1..5} -> route: 1-2-4-5 cost 4

distance-vector (Bellman-Ford/RIP) = neighbours exchange tables; link-state (OSPF) = flood + Dijkstra; count-to-infinity is the DV disease

Dijkstra step table proves OSPF converges to shortest paths (a favourite exam draw)

Algorithm	Information	Threshold	Drawback	Example
Flooding	none	n/a	exponential packets (good for resilience)	network discovery
Distance-vector	neighbour tables	Bellman-Ford equation	count-to-infinity, slow	RIP (<=15 hops)
Link-state	full map	Dijkstra	flood cost, heavyweight	OSPF, IS-IS
Path-vector	AS path lists	policy + path length	complex policy	BGP

THE UPDATE RULE BEHIND EACH ROUTING FAMILY

DV example: node A learns dist to X from B: $\text{distA}[X] = \min(\text{distA}[X], 1 + \text{distB}[X])$

LV flooding: A sends (id, seq) every link; neighbours rebroadcast unless seen -> OSPF DB

BGP: "tell my neighbours the AS path I use" -> loops die because path is explicit

4.4 IP Fragmentation & ICMP

FRAGMENTATION

Each link has an MTU (ethernet 1500). Too-big datagram = SPLIT at the router: identification field + offset (measured in 8-byte units) + MF (more fragments) allow reassembly. ICMP = control messages: echo (ping), destination-unreachable, TTL-expired (traceroute).

FRAG MATHS

Fragment 4000-byte payload for MTU 1500: 3 fragments of 1500,1480,1040... always compute offset = (prev_len)/8. Practice ONE.

Exercise: 1) In 192.168.10.0/24: usable hosts? broadcast? 2) Split it into 4 /26 subnets and list ranges. 3) Bellman-Ford one iteration on the Dijkstra graph. 4) A 4000-byte IP packet, MTU 1500: fragment offsets and MF bits.

UNIT 5 | TRANSPORT & APPLICATION LAYERS

5.1 Transport Services & UDP

TRANSPORT = END-TO-END MULTIPLEXING

PORTS choose the app on the far host; (IP:port) = the socket. UDP = connectionless, unreliable datagram (43 million IPs): low overhead, no handshake, no delivery guarantee - perfect for DNS lookups, DHCP, VoIP, streaming where drops cost less than delay.

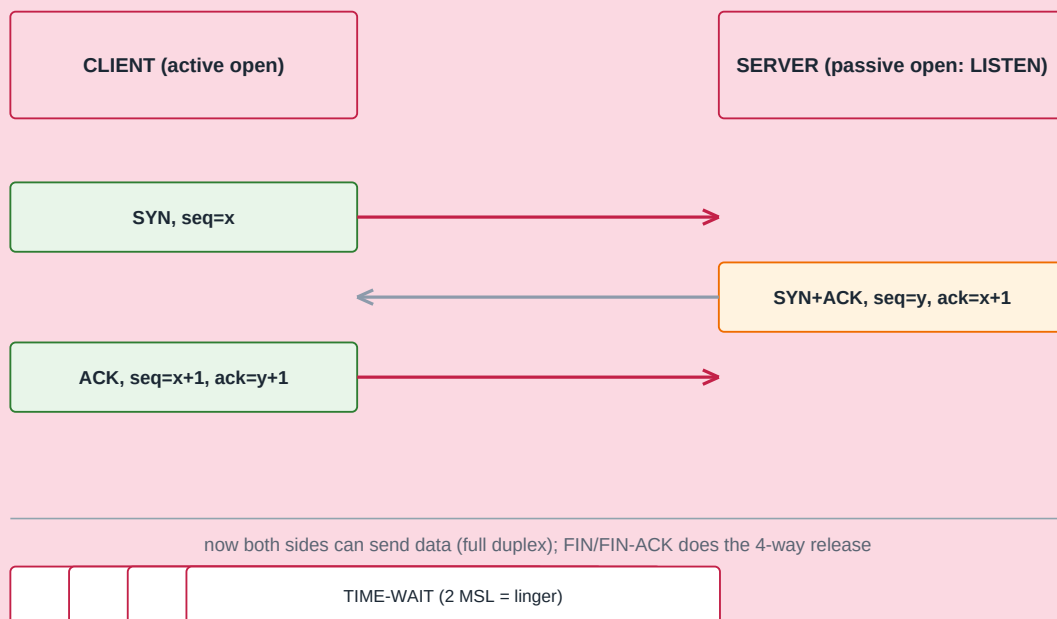
UDP vs TCP	UDP	TCP
Connection	none (each datagram independent)	3-way handshake, established session
Reliability	best effort only	seq + ACK + retransmission
Order	may reorder	in-order delivery
Overhead	8-byte header	20+ byte flags/options
Flow/congestion	no	window + AIMD
Typical apps	DNS, DHCP, RTP/WebRTC, QUIC (over UDP!)	HTTP, SMTP, FTP, SSH

ports.txt

```
# ports (estrange but official-ish examples):
ssh=22, http=80, https=443, DNS=53, DHCP=67/68, FTP=21/20, SMTP=25, POP3=110, IMAP=143, NTP=123
# see them live:
ss -tunp          # linux: all sockets with ports+programs  (netstat -ano on windows)
curl -v https://example.com  # watch the TLS+HTTP on top of TCP
```

5.2 TCP Internals

TCP: 3-Way Handshake + Connection States



segment header: src/dst port, seq, ack, flags, window, cksum, urgent (min 20 B)

flow control = advertised WINDOW; congestion = cwnd (AIMD, slow start 1->2->4...)

establish (SYN..ACK), data, release (FIN) - plus state names on the client side

TCP SEGMENT FIELDS THAT EXAMS QUOTE

src/dst PORT (16+16), sequence + ack numbers (32 each), data offset, flags (SYN/ACK/FIN/RST/PSH/URG), WINDOW (receiver buffer left), checksum, urgent pointer; options (MSS, window scale, SACK). Bytes are the unit - SEQ = byte number, ACK = next byte I expect.

RELIABILITY + EFFICIENCY MACHINERY

RETRANSMISSION TIMER = adaptive (RTT smoothed + deviation), so losses trigger resend even in bursts. FLOW CONTROL uses the receiver ADVERTISED WINDOW. CONGESTION CONTROL adds cwnd: SLOW START (double every RTT from 1) until ssthresh, then AIMD (add 1/RTT on success, halve on loss) - the sawtooth that saves the internet. Fast retransmit (3 dup-ACK) + fast recovery avoid needless timeouts.

TCP CONGESTION-CONTROL SAWTOOTH + THE NUMBERS

window math: time to send = window/MSS segments per RTT; throughput ~ window/RTT

slow start: cwnd = 1,2,4,8..., loss at 48 -> ssthresh=24, cwnd=1 grow again (or 24 with fast recovery)

real stacks: Cubic/Bbr variants fine-tune the probe - classic AIMD is the exam story

5.3 Application Layer Coast-to-Coast

DNS

Hierarchical distributed DB: root -> TLD (.com) -> authoritative; resolvers cache; records A (v4), AAAA (v6), MX (mail), CNAME, NS, TXT (SPF). Iterative + recursive resolution; "dig" the diagnostically. DNS translates BEFORE the browser ever dials TCP.

HTTP & EMAIL

HTTP/1.1 persistent + pipelined requests, request line / headers / body; status 1xx-5xx. HTTPS = TLS handshake over TCP (its own 3-burst cartoon). SMTP pushes MAIL (port 25/S 587) between MTA servers; POP3 (download) / IMAP (sync mailbox) let clients READ; MIME extends the body. FTP: control (21) + data (20) connections. SNMP monitors network devices via MIBs.

app_tools.txt

```
# application layer tools that resolve the round trip:
dig +trace vinay.com           # follow DNS from the root
curl -v http://example.com | head -n 20      # see request+response lines
openssl s_client -connect example.com:443 </dev/null 2>/dev/null | head -n 5  # TLS=HTTPS handshake
echo -e "HELO x\r\n" | nc smtp.gmail.com 587      # a raw SMTP greeting
```

EXAM SWEEP

Membership: one course usually ends with network vs transport vs application comparisons - nail the port table and the TCP-UDP table and you are home.

Exercise: 1) Draw the 3-way handshake with sequence numbers + then a 4-way close. 2) Trace "GET http://example.com/pic.jpg" naming ALL layers+protocols touched. 3) Slow start numbers when loss hits at 32. 4) If TCP is reliable, why does QUIC (UDP) exist?